



ASIAN
INSTITUTE OF
MANAGEMENT

Machine Learning 3

NLP using RNNs

Session 13 and 14

MSDS 2026

13 May 2026

ASIAN
INSTITUTE OF
MANAGEMENT

Class Administrative Matters

Course Deliverables

Mini Project 1 Starter Code

Course Schedule

Final Project Presentation (Wednesday June 10)



NLP Fundamentals II

Closure



NLP Fundamentals II

Proper Shape of H and C

After processing the sequence through the RNN, the hidden state serves as a compressed representation of the sequential information.

The hidden state's shape is predetermined by the user based on the desired **number of hidden** nodes.

```
# Candidate State
self.U_C = self.add_weight(
    shape=(num_inputs, num_hiddens),
    initializer=initializers.RandomNormal(stddev=sigma))
self.W_C = self.add_weight(
    shape=(num_hiddens, num_hiddens),
    initializer=initializers.RandomNormal(stddev=sigma))
self.b_C = self.add_weight(shape=(num_hiddens,), initializer="zeros")

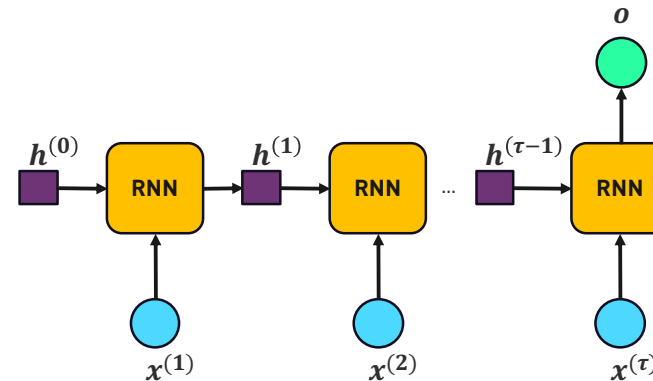
def call(self, inputs, H_C=None):
    if H_C is None:
        # Initial state with shape
        H = ops.zeros((self.num_hiddens,))
        C = ops.zeros((self.num_hiddens,))
    else:
        H, C = H_C
```

NLP Fundamentals II

MatMul Order

```
# Forward propagation
outputs = []
for X in inputs:
    for x in X:
        I = activations.sigmoid(
            ops.matmul(x, self.U_I) + ops.matmul(H, self.W_I) + self.b_I)
        F = activations.sigmoid(
            ops.matmul(x, self.U_F) + ops.matmul(H, self.W_F) + self.b_F)
        O = activations.sigmoid(
            ops.matmul(x, self.U_O) + ops.matmul(H, self.W_O) + self.b_O)
        C_tilde = activations.tanh(
            ops.matmul(x, self.U_C) + ops.matmul(H, self.W_C) + self.b_C)
        C = F*C + I*C_tilde
        H = O * activations.tanh(C)
    outputs.append(H)
```

$$h^{(t)} = g(Wh^{(t-1)} + Ux^{(t)} + b)$$



The order in which the vector and matrices are multiplied can be interchanged as long as proper dimensions have been defined for the weight matrices.

NLP Fundamentals II

Jena Climate Dataset TSF Model Performances

For the *reduced* dataset, the model performances of the different iterations of machine learning models is:

Model	MAE
Naive	2.76
Dense NN	2.75
1D CNN	3.45
LSTM	2.73
LSTM Dropout	2.63
Stacked GRU	2.58
Bidirectional RNN	2.77

Session 13 and 14

Session 13 and 14 – NLP Using RNNs

Gameplan

Session 11

NLP Fundamentals

Session 12

NLP using RNNs



NLP Fundamentals



NLP Fundamentals

What does “Natural Language” mean?

Any language that has been shaped by an *evolutionary process* rather than being artificially constructed like computer programming languages.



NLP Fundamentals

What are the challenges of Natural Language?

Natural language is messy – ambiguous, chaotic, sprawling, and constantly in flux.

NLP Fundamentals

What is Natural Language Processing (NLP)?

Natural language processing is an area of research in computer science and artificial intelligence (AI) concerned with processing natural languages such as English or Mandarin.

This processing involves translating language into data (numbers) that a computer can use to learn about the world.

And the understanding of the world is sometimes used to generate natural language text that reflects that understanding.



NLP Fundamentals

What are examples of NLP tasks?

Semantic Search

Analogy Problem Solving

Euphemism and Pun
Recognition

Paraphrase Recognition

Reading Comprehension

Intent Classification

Extractive Summarization

Humor and Sarcasm
Recognition

Sentiment Analysis

Question Answering

Logical Entailment and
Fallacy Recognition

Topic Modeling

Machine Translation

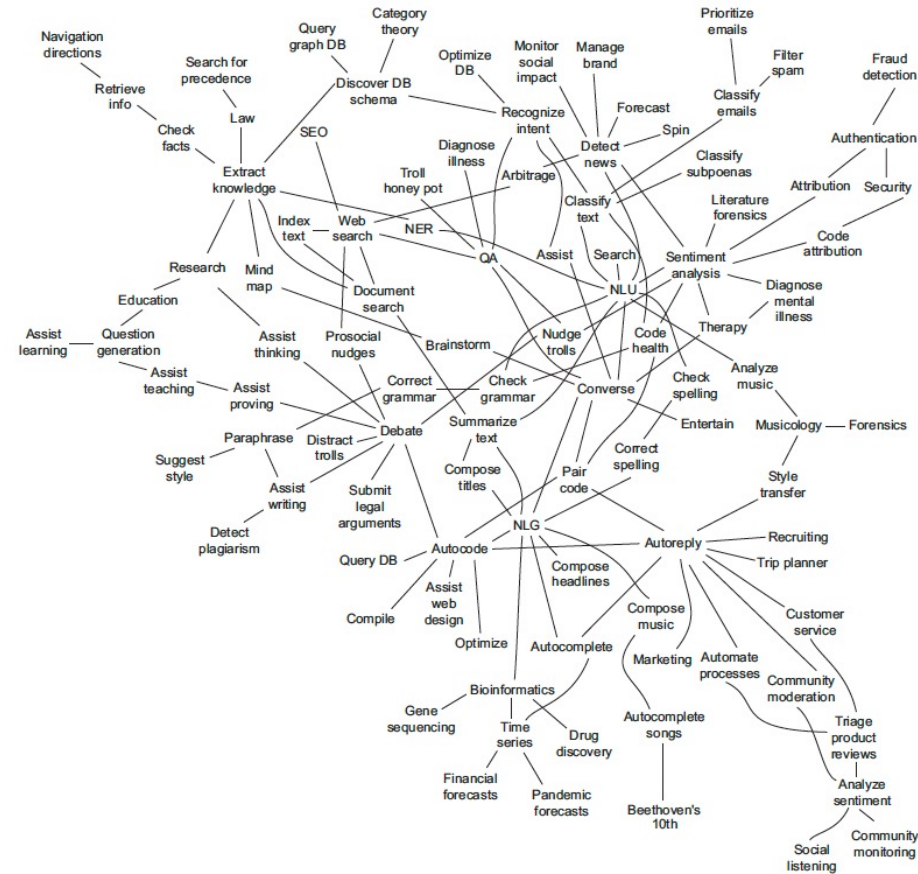
Knowledge Extraction

Authorship Attribution



NLP Fundamentals

NLP Applications



Lane, H., & Dyshel, M. (2019). Natural language processing in action. Simon and Schuster.

NLP Fundamentals

Getting Started with NLP: Data Preparation

Text Data Pre-processing Pipeline

1. **Tokenization and Case Folding** – the process of breaking down a sentence or a document into individual components or units called *tokens*. In some cases, the tokens are then converted to have the same case.
2. **Stemming, Lemmatization, Handling Contractions** – the process of extracting the *root word* of a given token.
3. **Stopwords Removal** – the removal of high-frequency words that can be ignored with minimal effect in accuracy.
4. **Word Vectorization** - the process of converting the pre-processed text data into vector form.



NLP Fundamentals

Tokenization

Character Tokenization

i love ml!

```
re.findall(r".", text)
```

```
["i", " ", "l", "o", "v",  
"e", " ", "m", "l", "!"]
```

Word Tokenization

i love ml!

```
re.findall(r"[\w]+|[.,!?;]", text)
```

```
["i", " ", "love", "ml", "!"]
```

NLP Fundamentals

Tokenization

Subword Tokenization by *byte-pair encoding*

Start with a basic vocabulary of characters and progressively “merge” common pairings into longer and longer sequences of characters.

```
data = [  
    "the quick brown fox",  
    "the slow brown fox",  
    "the quick brown foxhound",  
]
```



Worked Example – *byte-pair encoding*

```
data = [  
    "the quick brown fox",  
    "the slow brown fox",  
    "the quick brown foxhound",  
]
```

Count Pair of Tokens

Create Vocabulary of Words

Output

```
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]  
["t h e", "q u i c k", "b r o w n", "f o x", "s l o w", "f o x h o u n d"]
```



NLP Fundamentals

Text Vectorization

Order Agnostic (Sets)

Bag-of-Words

TF-IDF

Word2Vec

Order Aware (Sequence)

Integer Encoding

Doc2Vec

Hybrid

Positional Encoding



NLP using RNNs

NLP using RNNs

Representing Text as Numbers

Input text: “the cat sat on the mat”

One-hot encoding

	cat	mat	on	sat	the
the =>	0	0	0	0	1
cat =>	1	0	0	0	0
sat =>	0	0	0	1	0
...					

Integer encoding

```
mapping = {  
  "cat": 1, "mat": 2, "on": 3,  
  "sat": 4, "the": 5  
}  
  
input_text = [5, 1, 4, 3, 5, 2]
```

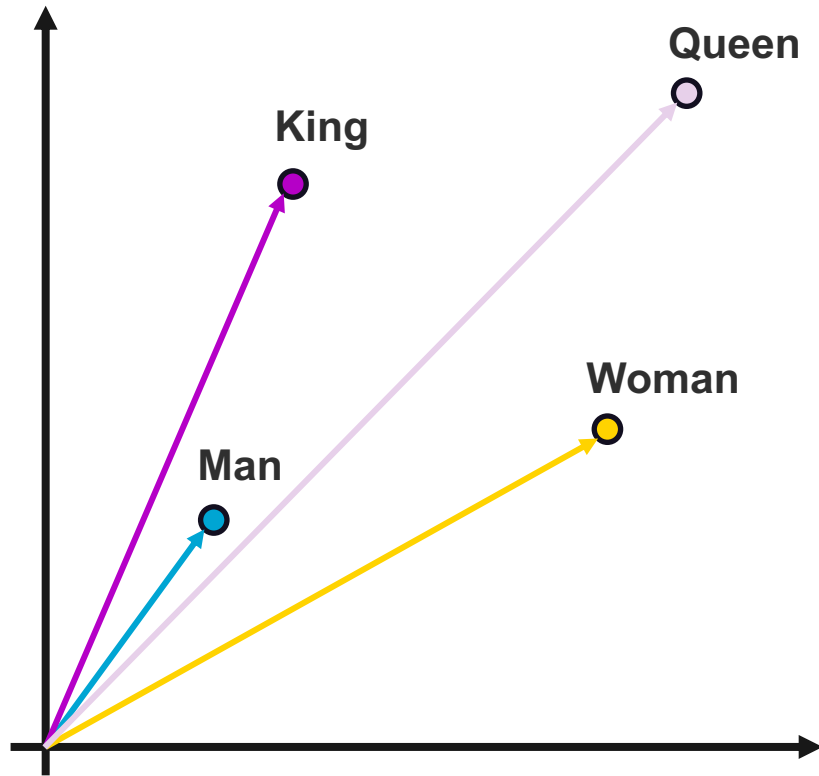
Word embeddings

cat =>	1.2	-0.1	4.3	3.2
mat =>	0.4	2.5	-0.9	0.5
on =>	2.1	0.3	0.1	0.4
...				

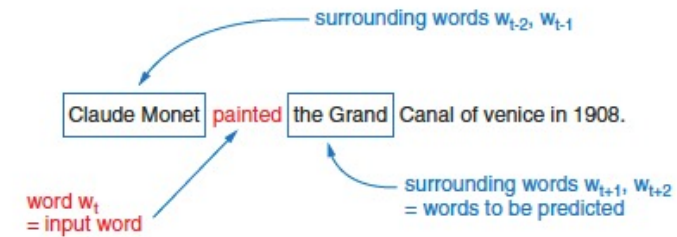


NLP using RNNs

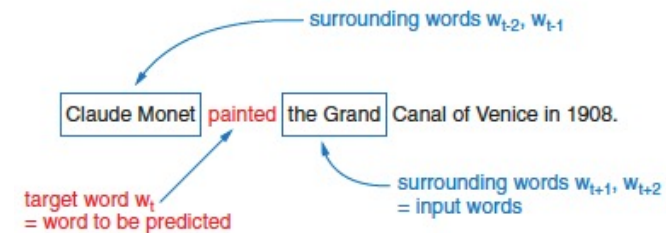
Word Embeddings



Skip-Gram



CBOW



NLP using RNNs

Word Embeddings

